

AIQG — Provisional Patent Filing Brief (Candidates 1-5)

Status: Draft for patent counsel — 2026-06-19. Technical drafting aid, **not legal advice**; claim scope and filing strategy require a patent attorney. **Inputs:** [AIQG-PATENT-ANALYSIS.md](#), [AIQG-AGENT-FLOW-ATTRIBUTION.md](#), [AIQG-AGENT-IDENTITY-RESEARCH.md](#). **FTO companion:** [AIQG-FTO-CLAIM-READ.md](#)

1. Title (working)

“Zero-Cooperation Attribution and Flow Reconstruction of AI-Agent Traffic at an LLM Gateway.”

2. Field

Observability, attribution, and security of multi-agent LLM systems at a network gateway / reverse proxy that mediates calls to one or more LLM providers.

3. The problem

A single AI-agent execution fans out into many independent LLM API calls (planner → workers, ReAct tool loops, RAG sub-queries) that span multiple processes, models, and toolsets. A gateway sees a flat stream of stateless chat-completion requests and cannot answer **“which agent, which run, which step produced this call?”** Existing approaches require the client to be instrumented — they propagate self-asserted identifiers in headers or SDK trace context (Helicone session headers, OpenTelemetry GenAI spans, LangSmith run ids). For **un-instrumented or third-party clients those mechanisms go dark**, and self-asserted ids are forgeable, so they cannot support security or billing-grade attribution.

4. Core insight (the thread tying the claims together)

The stateless chat-completions protocol **guarantees that the gateway’s own served output reappears in later requests**: the client must re-send the full conversation history every turn, and tool results must echo the exact `tool_call_id` values the gateway served. A gateway that indexes what it serves can therefore recognize it coming back and reconstruct agent topology **deterministically, with zero client cooperation** — the replay is forced by the API shape, not merely statistically likely. On top of that deterministic floor, agent *type* identity can be inferred statistically from the structural signals programmatic callers do not randomize.

5. Independent-claim concepts (the five candidates)

Candidate 1 — Content-propagation graph (strongest; design-only)

A method of attributing inter-agent data flow at an LLM gateway by: 1. content-defined chunking (e.g., Rabin fingerprinting, robust to byte shifts) of each **served** response into a fingerprint set; 2. storing those fingerprints in a TTL-bounded index keyed to the serving flow/step; 3. chunking each **incoming** request and looking its chunks up in the index; 4. on partial overlap, emitting a **data-flow edge** (parent flow → child sub-agent) — even across different processes, models, and toolsets where no header scheme could propagate anything; 5. **suppressing boilerplate via inverse-document-frequency weighting** so chunks common across many principals/templates (“You are a helpful assistant”, shared RAG corpus text) carry no attribution weight.

Why novel: the WAN-dedupe / byte-caching analogy (Bluecoat, Riverbed) is prior art in a different field; its application to **agent attribution in an LLM gateway with IDF boilerplate suppression** is absent from all surveyed prior art and from the four FTO patents.

Candidate 2 — Deterministic flow linkage via vendor-minted token echo

A method of reconstructing a multi-step agent flow by: 1. detecting `tool_calls` in a response the gateway serves and indexing each vendor-minted `tool_call_id` → (`flow_id`, `step_id`) in a short-TTL store; 2. on a later request carrying `role=tool` messages, matching the echoed `tool_call_id` to the index; 3. on match, asserting that the later request is the **next step of the same flow** and the indexed request is its **parent step**, yielding `flow_id` / `step_id` / `parent_step_id` for the full ReAct loop.

Why novel: the correlating token was minted by the vendor *through the gateway* — “the client cannot fake having received `call_abc123` from us” — so a match is **proof, not heuristic**. No surveyed tool does this.

Candidate 3 — Graded identity-confidence ladder with inference-audits-assertion security check

A system that, for each gateway event: 1. resolves identity top-down through an ordered ladder of sources (authenticated → asserted → baggage → linked → principal → transport → fingerprinted → behavioral → unattributed) and **stamps the winning identity_source plus a numeric identity_confidence**; 2. applies a **split-trust rule** — an *asserted* source wins for *who* (agent identity); a *linked* (evidence-based) source wins for *shape* (flow/step parentage); 3. when an asserted agent identity arrives on traffic whose **inferred** signature belongs to a different cluster, emits an **agent-impersonation / misconfiguration anomaly** — i.e., the inference tier audits the cooperative tier.

Why novel: individual tiers are routine; the **graded combination with a confidence stamp and inference-audits-assertion mismatch as a security signal** has no surveyed precedent. No other system can detect a client lying about which agent it is.

Candidate 4 — Fingerprint-ensemble agent registry with version lineage

A method of identifying agent *type* without client cooperation by: 1. extracting an ensemble of structural signals from each request (toolset hash, system-prompt template via log-template mining, user-message template, response_format JSON schema, few-shot blocks, scaffolding idioms, serialized-payload vocabulary, config tuple, step-archetype sequence, call-graph topology, token-count profile, periodicity, inter-step latency); 2. clustering in the joint signal space, **scoped per (tenant, principal)**, to a surrogate agent identity; 3. linking partially-changed signatures as a **new version of the same lineage** (MinHash Jaccard above threshold), so identity survives prompt iteration; 4. exposing clusters in a registry where **humans label clusters, not flows** (“14 agents detected, 3 named”), and where cooperatively tagged traffic doubles as **labeled seeds that calibrate the clustering** (semi-supervised).

Why novel: each technique borrows known art (Drain mining, MinHash, process mining); the **composition — per-(tenant,principal) ensemble + version lineage + human-labels-clusters registry + semi-supervised calibration from tagged traffic** — has no surveyed precedent.

Candidate 5 — Identity-ladder-keyed sticky experimentation on live gateway traffic

A system that runs controlled experiments (A/B) on live LLM-gateway traffic by: 1. assigning each request to a variant and keeping the assignment **sticky across a conversation/flow**

by **keying stickiness on the inferred identity ladder** — so un-instrumented callers, recognized only by the deterministic linkage tier or by fingerprint cluster, still receive **coherent, non-flip-flopping splits** they never opted into; 2. enforcing **claim-on-match mutual exclusion** so overlapping experiments do not contaminate each other’s cohorts; 3. computing a **non-inferiority verdict** on the decomposed CLEAR quality vector plus a strict objective win on cost/latency to decide ship/no-ship; 4. (*Gatekeeper-impact variant*) running **shadow pairwise evaluation** to prove the security layer’s rewrites (PII redaction, tokenization) do not degrade answer quality.

Why novel: gateway A/B testing exists; the differentiator is **assignment stickiness riding the zero-cooperation inferred identity ladder** — coherent experiment cohorts for callers that never opted in — combined with the Gatekeeper-impact non-inferiority design. Depends on Candidates 2-4: the identity ladder is the assignment key, which is what no instrumentation- dependent competitor can replicate.

6. Reduction to practice / enablement evidence

- **Shipped 2026-06-11 (aiqg-v5.24):** locked identity model, TAS-* + traceparent + baggage header contract, AgentContext event sub-struct, emitter field promotion, /api/v1/events, Traffic Explorer. Establishes the event schema and resolution-ladder plumbing the claims build on.
- **Shipped 2026-06-18 (Candidate 5):** the experiment Runner + verdict engine (cohort assignment, z-test non-inferiority verdict.go / significance.go, guardrails, Scout). Establishes reduction to practice for the experimentation substrate; the design-extension to file is **keying assignment stickiness on the inferred identity ladder** (rather than a self-asserted cohort id).
- **Design-only (to be built before/at filing):** the inference engine — tool_call_id echo + prefix chaining + content-propagation chunking + fingerprint ensemble + registry.
- **Validation artifact:** cmd/demo-traffic --untagged synthesizes multi-step agent profiles with known ground truth; running the inference tiers against it yields **precision/recall per tier** — the single best reduction-to-practice exhibit for the provisional. **Run this before filing.**

7. FTO posture (summary; see companion)

Claim-read of US 12,483,411 (Tyntre, blockchain agent identity), US 12,547,677 (Airia, JIT per-user tool auth), US 10,924,326 (Dynatrace, distributed-trace correlation), US 11,785,115 (IBM, socket-correlation request tracing) found **no HIGH-risk read** on any of the four candidates as described. The content-propagation graph (Candidate 1) is **clear** of all four. Dynatrace 10,924,326 is the one to take most seriously (broad correlation/metering claims) and warrants a counsel-led read before billing-grade metering. Design-arounds documented in the FTO companion.

8. Recommended filing scope and sequence

1. **File one provisional covering Candidates 1-5 as a family** before any public disclosure — Candidates 1-4 share the “gateway recognizes its own served output” core, and Candidate 5 consumes that same inferred-identity ladder as its experiment-assignment key, so all five are strongest claimed together. Lean provisional over defensive publication for these moat-bearing pieces.
2. **Before filing:** run cmd/demo-traffic --untagged, capture per-tier precision/recall, and fold the numbers + architecture diagrams into the provisional as enablement.
3. **In parallel, counsel-led FTO read** focused on Dynatrace 10,924,326 and the tool_call_id-echo / per-agent-metering techniques before any billing-grade use.

4. **Keep CLEAR scoring** and the Scout/autolearning ML candidates (see [AIQG-PATENT-SCOUT-AUTOLEARNING.md](#), [AIQG-PATENT-ML-IN-LOOP.md](#)) for a later, separate filing decision — out of scope here, pending the bandit/ auto-experimentation prior-art search.

9. Disclosure hygiene (statutory-bar guard)

The inferred-attribution design currently exists only in private internal repos/docs — no statutory bar has started. **Do not publicly disclose, demo, or publish the inference design** (talks, blog posts, open-sourcing the inference engine, investor materials beyond NDA) until the provisional is on file.